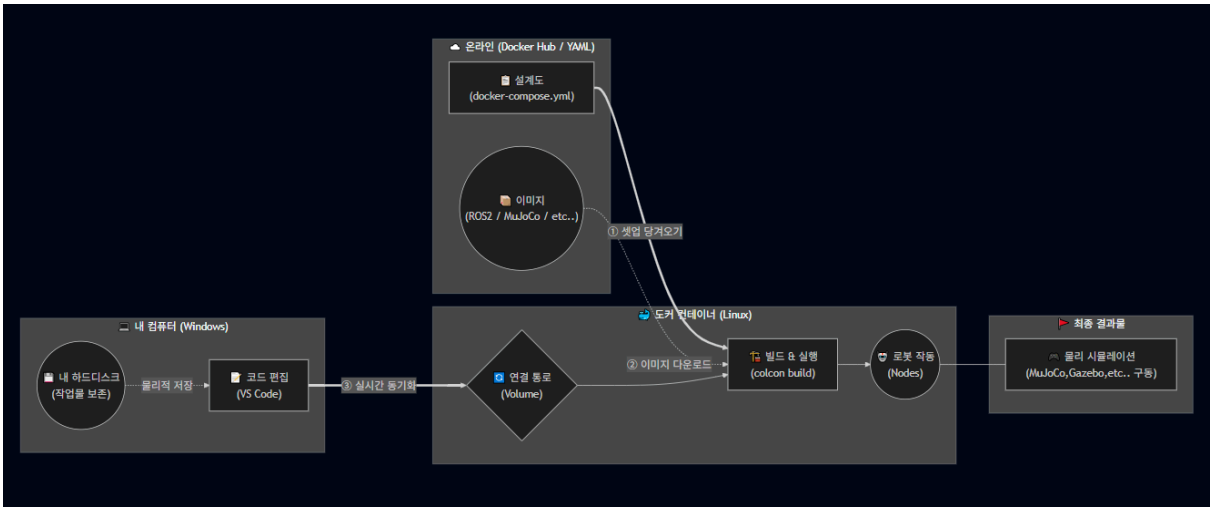
# Docker 정리본

## 🔍 *도커 핵심 개념 정리*

| 구분 | 설명 |
|---|---|
| **이미지 (Image)** | 실행에 필요한 모든 파일이 담긴 **읽기 전용 압축 파일** |
| **컨테이너 (Container)** | 이미지를 격리된 환경에서 **실제로 실행시킨 상태** |
| **볼륨 (Volume)** | 컨테이너 삭제 후에도 **데이터가 유지되는 외부 저장소** |

## 예시 이미지




| 단계 | 핵심 구성 요소 | 주요 역할 (설명) |
|---|---|---|
| **1. 준비 (Pull)** | **설계도 & 부품** | 인터넷에서 `docker-compose up`으로 필요한 로봇 환경(이미지)을 내 컴퓨터로 통째로 가져옴. |
| **2. 작업 (Volume)** | **내 컴퓨터 (Host)** | **VS Code**로 코드를 수정하면, **연결 통로인 Volume을** 통해 도커 실험실에 실시간으로 반영되고 안전하게 보관. |
| **3. 실행 (Run)** | **도커 컨테이너** | 도커 안에서 코드를 빌드하고 로봇(Node)을 깨워 **MuJoCo/Gazebo 등** 가상 세계에서 실제로 작동. |

** 명령어 입력 전에 항상 Docker를 켜고 진행할 것.*

---

## 🛠️ 도커 기본 명령어

```
# 1. 이미지 관리
docker pull [이미지명]        # 도커 허브에서 이미지 다운로드 ex.
docker pull osrf/ros:humble
docker images                 # 내 PC에 있는 이미지 목록 확인
docker inspect [이름/ID]      # 컨테이너/이미지의 상세 정보(IP,
설정 등) 확인

# 2. 실행 및 상태 확인
docker run [이미지명]         # 이미지를 컨테이너로 생성 + 실행
docker ps                     # 현재 실행 중인 컨테이너 목록
docker ps -a                  # 중지된 컨테이너를 포함한 전체 목
록 확인
docker logs [컨테이너ID]      # 컨테이너 내부 출력 로그 확인
docker logs -f [컨테이너ID]   # 특정 컨테이너의 로그를 실시간으로
계속 출력 (-f: 실시간 추적)

# 3. 제어 및 삭제
docker stop [컨테이너ID]      # 실행 중인 컨테이너 중지
docker start [컨테이너ID]     # 중지된 컨테이너 다시 시작
docker rm [컨테이너ID]        # 컨테이너 삭제 (중지 상태여야 함)

# 4. 내부 명령 실행
docker exec -it [컨테이너ID] bash   # [작동 중인] 컨테이너 내부
로 접속하여 터미널(bash) 실행
```

** 대괄호는 제외하고 작성할 것*

---

## 🛠️ 도커 핵심 명령어 요약

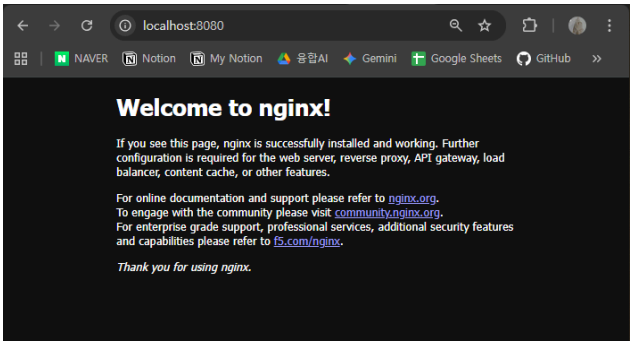
### 1) 웹 서버 실행 (포트 연결)

```
docker run -d -p 8080:80 --name web nginx
```

- **해석:** 내 컴퓨터 **8080포트**와 컨테이너 **80포트**를 연결하여 Nginx 서버를 백그라운드로 실행함.
- 포트포워딩(-p) 안 쓰면 외부접속이 아예 불가능

👉 **접속:** `http://localhost:8080`

- local host 접속 예시 이미지




---

## 2) OS 컨테이너 백그라운드 유지

```
docker run -it -d --name ubuntu ubuntu:22.04 bash
```

- **해석:** 우분투를 백그라운드에서 꺼지지 않게 '대기 상태'로 켜둠.
- **핵심:** `-d`를 써서 뒷마당(백그라운드)에서 서버가 계속 돌게 함. `-it` 덕분에 "사용자가 언제든 들어올 수 있게 터미널 준비하고 대기해!"라는 상태가 됨.

👉 뒤에 bash 붙이면 바로 터미널창 열리게 함.

---

## 3) 실행 중인 컨테이너 접속 (들어가기)

```
docker exec -it ubuntu bash    # execute : 실행하다의 약어.
```

- **해석:** 이미 켜져 있는(run 상태) 컨테이너 안으로 직접 들어감. (예시는 ubuntu라는 컨테이너)
- **비유:** `run`은 새 집 짓기, `exec`은 지어진 집 방문하기임.

---

## 4) 내 파일 공유 (볼륨 마운트)

```
docker run -it --name my_ros -v C:/work/docker_ws:/ros_ws u
```

```
buntu:22.04 bash
```

- **해석: [내 컴퓨터 폴더]:[컨테이너 폴더]** 를 하나로 연결.
- **장점:** 컨테이너를 지워도 내 컴퓨터 하드에 작업물이 남음. (ROS 개발 필수)

---

## 📋 *docker-compose*

### 1) `docker-compose.yaml` (설계도)

- 긴 명령어를 칠 필요 없이 `docker-compose.yml` 파일 하나로 여러 컨테이너를 일괄 관리함.

```yaml
version: "3.8"

services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

- **내용:** 이미지 이름, 포트 번호 등 실행에 필요한 모든 정보를 적어둔 파일임.
- **장점:** 긴 명령어를 매번 칠 필요 없이 파일 하나로 관리함.

👉 docker-compose.yaml 파일은 띄어쓰기에 굉장히 예민함. *(클론, 대시 뒤에는 띄어쓰기로 구분해주기.)*

---

### 2) 주요 명령어

```
# 1. 일괄 관리
docker compose up          # 설정파일(yaml)대로 모든 서비스 생성 및 실행
docker compose up -d       # (서버 켜기) 백그라운드 모드로 실행 (터미널 계속 사용 가능)
docker compose down        # (서버 닫기)모든 서비스 중지 및 생성된 컨테이너 삭제
```

```
# 2. 빌드 및 확인
docker compose build       # Dockerfile 수정 시 이미지를 새로 다시 빌드
docker compose ps          # Compose로 관리되는 서비스들의 상태 확인
docker compose logs -f     # 모든 서비스의 로그를 실시간으로 모니터링(-f)

# 3. 특정 서비스 제어
docker compose exec [서비스명] [명령어]  # 특정 컨테이너 안에 들어가 명령 실행
```

---

## 📰 심화 docker-compose.yaml 파일 형식

```
services:
  ros2:
    image: osrf/ros:humble-desktop   # osrf에서 ros humble과 관련 기본 파일들 가져옴.
    container_name: ros2_container   # 여기에 원하는 이름 작성
    stdin_open: true                 # 터미널 입력 열어두기
    tty: true                        # 컨테이너가 안 꺼지게 대기시키기

    volumes:
      - ./workspace:/ros_ws          # [내 컴퓨터 폴더]:[컨테이너 폴더] 연결

    working_dir: /ros_ws             # 접속 시 시작할 폴더 지정

    command: bash                    # 시작하자마자 터미널(bash) 실행
```

*cf) osrf : Open Source Robotics Foundation : ROS를 만드는 공식 단체 계정*

💡 꼭 docker-compose.yaml 파일이 있는 곳에서 실행시켜줘야 함.

```
docker-compose up -d            # 모든 컨테이너를 한 번에 생성하고 서버실행 (백그라운드 모드)
docker-compose exec ros2 bash   # 실행 중인 'ros2' 서비스 내부로 들어가서 터미널(bash)을 실행함
```

** 컨테이너ID를 입력하는게 아니라, Services 이름을 입력해야 함.*

---

## 📦 *docker-compose 이후, ROS2 패키지 생성 및 빌드 과정 요약*

### 1) ROS2 환경 확인

```
source /opt/ros/humble/setup.bash   # 터미널 열 때마다 해야하기 때문에 ~/.bashrc에 넣어줘야 함.
ros2 --version
```

---

### 2) 패키지 생성

```
ros2 pkg create --build-type ament_python my_pkg
```

- **해석:** Python 기반의 ROS2 패키지(`my_pkg`)를 자동으로 만들어줌.
- **결과:** `setup.py`, `package.xml` 등 필수 파일들이 담긴 폴더가 생성됨.

---

### 3) 빌드 실행

```
colcon build
```

- **명령:** `/workspace` 경로에서 실행함.
- **해석:** 내가 만든 소스코드(`src`)를 컴퓨터가 실행할 수 있는 형태로 변환(컴파일/설치)함.
- **결과:** `build`, `install`, `log` 폴더가 자동으로 생김.

- `install/`: 실제로 실행될 파일들이 모이는 곳임.
- `build/`: 빌드 과정 중 발생하는 임시 파일 저장소임.

---

### 💡 빌드 후 필수 단계 (중요)

빌드가 끝났다고 바로 실행할 수 있는 게 아님. **내가 만든 패키지를 인식**시켜줘야 함.

```
# /workspace 폴더에서 실행함
source install/setup.bash
```

- **해석:** "방금 빌드해서 `install` 폴더에 새로 생긴 패키지들을 현재 터미널에 등록해라"라는 뜻임.
- **순서:** `colcon build` ➔ `source install/setup.bash` ➔ `ros2 run ...` (이게 한 세트임)

👉 **주의** : src/my_pkg 여기서 노드를 만들어야 하는데, install/my_pkg에서 할 수도 있음.

---

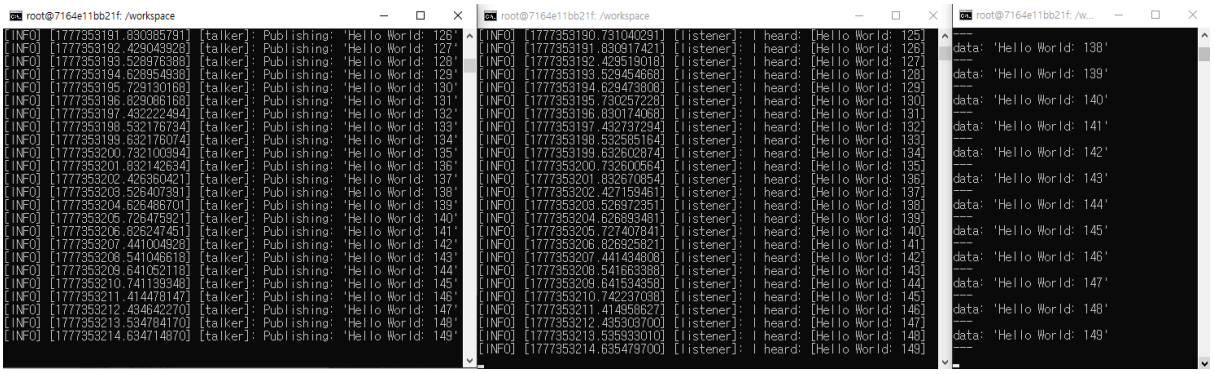
## 4) talker / listner 예제 노드 (ROS 기본 제공 - 확인용)

```
## 1번째 터미널창
ros2 run demo_nodes_cpp talker

## 2번째 터미널창
ros2 run demo_nodes_cpp listener

## 3번째 터미널창
ros2 topic echo /chatter    # echo = Linux에서 토픽 세부 내용들을 터미널로 출력하는 명령어
```

**결과 이미지 (1. talker / 2. listener / 3. echo chatter)**




---

## 5) 실제 노드 생성

### 파일 경로

```
workspace/
├── build/                    # 빌드 결과물이 생성되는 곳 (건드리지 않음)
├── install/                  # 실행 파일이 설치되는 곳 (건드리지 않음)
├── log/
└── src/                      # 💡 우리가 작업하는 소스 폴더
    └── my_pkg/
        ├── package.xm
        ├── setup.py          # 빌드 및 엔트리포인트 설정 파일
        ├── setup.cf
        └── my_pkg/           # 📍 실제 파이썬 코드가 담기는 곳
            ├── __init__.py
            └── my_node.py    # ⭐ 여기에 파일을 생성
```

### my_node.py

```python
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
```

```
        super().__init__('my_node')
        self.get_logger().info("Hello ROS2")

def main():
    rclpy.init()
    node = MyNode()
    rclpy.spin(node)
    rclpy.shutdown()
```

**setup.py**

```
from setuptools import find_packages, setup

package_name = 'my_pkg'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='root',
    maintainer_email='root@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
```

```
            'my_node = my_pkg.my_node:main',    # [추가] -
새로운 노드 추가
        ],
    },
)
```

**빌드 및 실행**

```
cd /workspace/
colcon build
source install/setup.bash    # 업데이트한 걸 source로 알려줘야
함.

ros2 run my_pkg my_node      # 노드 실행
```

---

▼ *docker-compose 응용 (MySQL / ROS2 / MuJoCo)*

AI 프롬프트

```
" 너는 20년차 로보트 시스템 전문 아키텍처야. "
" docker-compose 기반으로 MySQL 8.0 / ROS2 Humble / MuJoC
o 3.6.0 서버를 구성해줘. "
" 초보자도 이해하기 쉽도록 주석문을 사용하여 구성 내역에 대한 상
세한 설명을 같이 작성해줘. "
```

생성된 docker-compose.yaml

```yaml
version: '3.8'

services:
  # 1. 데이터베이스 서버: 로봇의 기록을 담당합니다.
  db:
    image: mysql:8.0
    container_name: robot_database
    restart: always
    environment:
```

```
      MYSQL_DATABASE: robot_logs
      MYSQL_ROOT_PASSWORD: password123 # 실제 사용 시 보안을 위해 변경하세요.
    ports:
      - "3306:3306"
    volumes:
      - ./mysql_data:/var/lib/mysql # 컨테이너가 꺼져도 데이터가 보존되도록 설정

  # 2. 로봇 두뇌 (ROS2 Humble): 통신과 제어를 담당합니다.
  ros2-humble:
    image: osrf/ros:humble-desktop
    container_name: ros2_master
    stdin_open: true
    tty: true
    network_mode: "host" # ROS2의 원활한 멀티캐스트 통신을 위해 호스트 네트워크 권장
    volumes:
      - ./src:/root/ros2_ws/src # 내 소스코드를 컨테이너와 동기화
    environment:
      - DISPLAY=$DISPLAY # GUI 도구(Rviz 등) 사용을 위한 설정
    depends_on:
      - db # DB가 먼저 켜진 후 실행되도록 설정

  # 3. 물리 엔진 (MuJoCo 3.6.0): 가상 세계를 담당합니다.
  mujoco:
    build:
      context: .
      dockerfile: Dockerfile.mujoco
    container_name: mujoco_sim
    runtime: nvidia # GPU 가속이 필요한 경우 (NVIDIA Docker 필요)
    environment:
      - DISPLAY=$DISPLAY
    volumes:
      - ./models:/root/.mujoco/models # 로봇 모델 파일(.xm
```

```
l) 경로
    depends_on:
      - ros2-humble
```